



Contenerización con Docker

De cero a tu app en un contenedor

Paso a paso · WSL · Ubuntu ·

Módulo 2 · Sesión: Contenedorización con Docker

 Proyecto listo	 Dockerfile	 .dockerignore	 docker build	 Docker Hub
Paso 1	Paso 2	Paso 3	Paso 4–5	Paso 6–9

PARTE 0 — Instalar Docker en Ubuntu / WSL

Si ya tienes Docker instalado y funcionando, salta directamente a la Parte 1.
Compruébalo con: `docker --version`

PASO 0.1 Preparar el entorno

```
sudo apt update
sudo apt install apt-transport-https ca-certificates curl gnupg
lsb-release
```

PASO 0.2 Añadir la llave GPG oficial de Docker

Esto verifica que los paquetes que descargas son auténticos:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg
--dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

PASO 0.3 Configurar el repositorio estable

```
echo "deb [arch=$(dpkg --print-architecture)
signed-by=/usr/share/keyrings/docker-archive-keyring.gpg]
https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" |
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

PASO 0.4 Instalar Docker Engine

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

PASO 0.5 Verificar que el servicio está activo

```
sudo systemctl status docker
```

✓ Si ves: **active (running)** en verde — Docker está listo.

PASO 0.6 Evitar escribir sudo en cada comando (opcional)

Por defecto Docker requiere sudo. Para evitarlo, añade tu usuario al grupo docker:

```
sudo usermod -aG docker $USER
```

⚠ **Cierra y vuelve a abrir la terminal para que el cambio tenga efecto.**
Desde ese momento puedes usar docker sin sudo.

PASO 0.7 Test definitivo de la instalación

```
docker run hello-world
```

✓ Si ves el mensaje 'Hello from Docker!' — todo está funcionando.

Lo que acaba de pasar: Docker descargó una imagen de Docker Hub y la ejecutó.
Ya viste el ciclo completo: pull (descargar) → run (ejecutar contenedor).

Punto de Partida — Lo que ya tienes

Ya tienes una app de Streamlit funcionando en tu máquina. Ese es tu punto de partida. No necesitas cambiar una sola línea de código.

Estructura inicial de tu proyecto

```
mi_app/
├── app.py           ← tu aplicación Titanic con Streamlit
├── pages/           ← tus páginas (si tienes estructura
multipágina)
│   ├── 1_Pagina.py
│   └── 2_Pagina.py
└── requirements.txt ← lista de dependencias
```

⚠ Verifica tu **requirements.txt** ANTES de continuar:

Debe contener TODAS las librerías que usa tu app.

Si tu app importa pandas, plotly, seaborn... tienen que estar aquí.

Docker instalará las dependencias desde CERO usando solo este archivo.

Si falta alguna librería aquí, la app fallará dentro del contenedor.

Ejemplo de requirements.txt correcto

```
streamlit>=1.30.0
pandas>=2.0.0
plotly>=5.0.0
seaborn>=0.13.0
numpy>=1.24.0
```

PARTE 1 — Verifica que tu Proyecto está Listo

Antes de crear ningún archivo de Docker, asegúrate de que tienes la estructura correcta y que el requirements.txt está completo.

PASO 1 Navega a tu carpeta de proyecto

Tus archivos de Windows son accesibles desde WSL bajo /mnt/. Ejemplo:

```
cd /mnt/c/Users/TU_USUARIO/.../mi_app
```

La ruta mágica de WSL:

C:\Users\tu_user\...\mi_app → /mnt/c/Users/tu_user/...\mi_app

Cada unidad de disco Windows aparece bajo /mnt/ (C: → /mnt/c, D: → /mnt/d)

PASO 2 Verifica la estructura del proyecto

```
ls -la
```

Deberías ver exactamente estos archivos:

```
app.py
pages/
requirements.txt
```

⚠ Si ves Dockerfile.txt en lugar de Dockerfile, renómbralo:

mv Dockerfile.txt Dockerfile

Windows añade .txt automáticamente — hay que eliminarlo.

PASO 3 Verifica el contenido de requirements.txt

```
cat requirements.txt
```

Debe contener todas las librerías que usa la app:

```
streamlit>=1.30.0
pandas>=2.0.0
plotly>=5.0.0
seaborn>=0.13.0
numpy>=1.24.0
```

¿Por qué esto importa?

El contenedor NO tiene acceso a tu venv/. Docker instala las dependencias desde cero usando únicamente este archivo. Si falta algo aquí, la app fallará dentro del contenedor aunque funcione en tu máquina.

PARTE 2 — Crear el .dockerignore

Creamos el .dockerignore ANTES que el Dockerfile porque es lo primero que Docker consulta al copiar archivos. Igual que .gitignore le dice a Git qué archivos no versionar, .dockerignore le dice a Docker qué archivos NO copiar al construir la imagen.

Analogía con Git:

.gitignore → archivos que Git no sube al repositorio remoto
.dockerignore → archivos que Docker no mete dentro del contenedor
Mismo concepto, distinto contexto.

PASO 4 Crear el .dockerignore desde WSL

Ejecuta este comando tal cual — crea el archivo con todo el contenido en un solo paso:

```
cat > .dockerignore << 'EOF'
venv/
__pycache__/
*.pyc
*.pyo
.git/
.gitignore
*.zip
*.md
EOF
```

Verifica que se creó (los archivos con punto no se ven con ls normal):

```
ls -la
```

¿Por qué ignorar venv/?

El entorno virtual de tu máquina contiene cientos de MB de archivos compilados específicos para tu sistema operativo. El contenedor usa Linux y los ignoraría. Docker instala sus propias dependencias con pip install dentro del contenedor. Incluir venv/ haría la imagen enorme e inútil.

¿Por qué excluimos cada cosa?

Archivo/Carpeta	Por qué se excluye
EOF : End Of File - "fin del archivo"	Es una convención de Linux/Bash para decirle al terminal: <i>"todo lo que escriba a partir de aquí hasta que vuelva a ver EOF es el contenido del archivo"</i> .
venv/	Contiene cientos de MB compilados para TU sistema. Docker usa Linux e instala sus propias dependencias. Incluirlo haría la imagen enorme e inútil.
__pycache__/	Archivos compilados de Python. Se regeneran solos. No aportan nada al contenedor.
*.pyc / *.pyo	Lo mismo — archivos compilados temporales de Python.
.git/	El historial de Git no tiene ningún sentido dentro del contenedor.

Archivo/Carpeta	Por qué se excluye
*.zip / *.md	Archivos de documentación y comprimidos que no necesita la app para funcionar.

PARTE 3 — Crear el Dockerfile

El Dockerfile es una receta de cocina. Docker la sigue paso a paso de arriba a abajo para construir tu imagen.

Cada instrucción crea una capa. Las capas que cambian poco van arriba. Las que cambian frecuentemente van abajo.

Analogía con Git:

El README.md describe cómo instalar y usar tu proyecto.
 El Dockerfile describe cómo construir y ejecutar tu entorno.
 Pero con una diferencia clave: el README lo lee un humano.
 El Dockerfile lo lee Docker — y lo ejecuta automáticamente.

Concepto clave — Capas

Cada instrucción del Dockerfile crea una CAPA en la imagen. Piénsalo como los commits de Git:

1. Cada commit guarda el estado del código en ese momento.
2. Cada capa guarda el estado del sistema en ese momento.
3. Docker puede reutilizar capas anteriores si no han cambiado (igual que Git no re-sube lo que no cambió).

La regla de oro del orden:

Las instrucciones que cambian POCO van ARRIBA (se cachean).
 Las instrucciones que cambian FRECUENTEMENTE van ABAJO.
 → Si solo cambias tu código, Docker reutiliza todas las capas anteriores.
 → Resultado: builds mucho más rápidos a partir del segundo.

PASO 5 Crear el Dockerfile

Crea el archivo Dockerfile (sin extensión, con D mayúscula):

touch Dockerfile # crea el archivo vacío

nano Dockerfile # lo abres para editarlo y escribes lo siguiente:

```
# — 1. IMAGEN BASE —————
# Partimos de Python 3.11 en su versión ligera (sin extras).
# NUNCA uses :latest — la versión puede cambiar y romper la app.
FROM python:3.11-slim
```

```

# — 2. DIRECTORIO DE TRABAJO —————
# Crea la carpeta /app dentro del contenedor y se mueve a ella.
# Todo lo que siga ocurrirá dentro de /app.
WORKDIR /app

# — 3. COPIAR DEPENDENCIAS (ANTES que el código) —————
# Truco clave de caché: si el código cambia pero el .txt no,
# Docker salta los pasos 3 y 4 usando caché → build más rápido.
COPY requirements.txt .

# — 4. INSTALAR DEPENDENCIAS —————
# --no-cache-dir: no guarda caché de pip → imagen más pequeña.
RUN pip install --no-cache-dir -r requirements.txt

# — 5. COPIAR EL RESTO DEL PROYECTO —————
# Ahora sí copiamos app.py, pages/, etc.
# Va DESPUÉS de pip install para aprovechar la caché.
COPY . .

# — 6. EXPONER EL PUERTO —————
# Streamlit usa el 8501. EXPOSE solo documenta — no abre el puerto.
# El puerto se abre en docker run con -p.
EXPOSE 8501

# — 7. COMANDO DE ARRANQUE —————
# Lo que se ejecuta cuando el contenedor arranca.
# Formato JSON obligatorio: ["cmd", "arg1", "arg2"]
# 0.0.0.0 = accesible desde fuera del contenedor (imprescindible).
CMD ["streamlit", "run", "app.py",
    "--server.address=0.0.0.0",
    "--server.port=8501"]

```

El orden de las instrucciones NO es aleatorio — es una decisión técnica:

Instrucciones 1-2: configuración del entorno (nunca cambian)
 Instrucciones 3-4: dependencias (cambian solo si añades librerías)
 Instrucción 5: tu código (cambia en cada desarrollo → va al final)
 → Así Docker reutiliza la caché al máximo en cada rebuild.

Error muy común en Windows:

Windows guarda el archivo como 'Dockerfile.txt' en lugar de 'Dockerfile'.
 Si esto ocurre, renómbralo desde WSL:
 mv Dockerfile.txt Dockerfile

Verificamos que se se haya creado correctamente:

```
cat Dockerfile
```

PARTE 4 — Verifica la Estructura Final

Antes de lanzar el build, comprueba que tienes los siguientes archivos necesarios en la carpeta:

PASO 6 Verificar que todos los archivos están presentes

```
ls -la
```

Deberías ver exactamente:

```
mi_app/
├── app.py      ← tu aplicación Streamlit
├── pages/     ← en caso de que existan OPCIONAL
│   ├── 1_Explorador.py
│   ├── 2_Graficos.py
│   └── 3_Acerca_de.py
├── requirements.txt ← lista de dependencias
├── Dockerfile   ← añadir esto. sin extensión, D mayúscula
├── .dockerignore ← añadir esto, empieza con punto
└── README.md    OPCIONAL
```

Errores más comunes en este punto:

- Dockerfile.txt en lugar de Dockerfile → `mv Dockerfile.txt Dockerfile`
- .dockerignore no aparece con `ls` → normal, usa `ls -la` para verlo
- No estás en la carpeta correcta → comprueba con `pwd`

Comandos esenciales de gestión

Comando	¿Qué hace?
<code>docker ps</code>	Ver contenedores en ejecución
<code>docker ps -a</code>	Ver TODOS los contenedores (incluidos parados)
<code>docker images</code>	Ver las imágenes construidas/descargadas
<code>docker stop <id></code>	Parar un contenedor en ejecución
<code>docker rm <id></code>	Eliminar un contenedor parado
<code>docker rmi <imagen></code>	Eliminar una imagen local
<code>docker run -d -p 8501:8501 </code>	Ejecutar en segundo plano (modo detached)
<code>docker logs <id></code>	Ver los logs de un contenedor

PARTE 5 — Construir y Ejecutar el Contenedor

PASO 7 Construir la imagen

```
sudo docker build -t titanic-explorer .
```

Desglose del comando — igual que un commit en Git:

`docker build` → construye la imagen (como git commit)
`-t mi-app-streamlit` → nombre que le das a la imagen (como el mensaje del commit)
`.` → usa la carpeta actual (donde está el Dockerfile)

Verás cómo Docker ejecuta cada paso en orden:

```
Step 1/7 : FROM python:3.11-slim
Step 2/7 : WORKDIR /app
Step 3/7 : COPY requirements.txt .
Step 4/7 : RUN pip install --no-cache-dir -r requirements.txt
Step 5/7 : COPY . .
Step 6/7 : EXPOSE 8501
Step 7/7 : CMD [...]
Successfully built alb2c3d4e5f6
Successfully tagged titanic-explorer:latest
```

✓ Cuando ves 'Successfully tagged' — la imagen está lista.

La primera vez tarda más (descarga Python ~120 MB).

Las siguientes veces es mucho más rápido gracias a la caché de capas.

PASO 8 Verificar que la imagen se creó

```
sudo docker images
```

Deberías ver `titanic-explorer` en la lista con su tamaño y fecha.

PASO 9 Ejecutar el contenedor

```
sudo docker run -p 8501:8501 titanic-explorer
```

Desglose del comando:

`docker run` → crea y arranca un contenedor
`-p 8501:8501` → mapea el puerto (LOCAL:CONTENEDOR)
`titanic-explorer` → nombre de la imagen a usar

Cuando veas esto en la terminal:

```
You can now view your Streamlit app in your browser.
URL: http://0.0.0.0:8501
```

✓ Abre el navegador en: <http://localhost:8501>

Tu app del Titanic está corriendo dentro de un contenedor Docker. 🎉

Para parar el contenedor:

```
Ctrl + C
```

Para ejecutar en segundo plano (sin bloquear la terminal):

```
sudo docker run -d -p 8501:8501 --name titanic-app titanic-explorer  
sudo docker stop titanic-app # para pararlo después
```

PARTE 6 — Subir la Imagen a Docker Hub

Ahora viene la parte equivalente al git push. Docker Hub es el 'GitHub de las imágenes Docker'. Es público, gratuito y es donde vive la imagen de Python que usamos en el FROM. Vas a subir la imagen a Docker Hub para que otra/o coder pueda descargarla y verificarla.

Analogía con Git:

git push origin main → subes tu código a GitHub
docker push usuario/mi-app:v1.0 → subes tu imagen a Docker Hub
En ambos casos, alguien más puede luego descargarlo con pull.

PASO 10 Crear cuenta en Docker Hub

4. Ve a <https://hub.docker.com> y crea una cuenta gratuita.
5. Anota tu username — lo usarás en los comandos que siguen.
6. Ve a tu perfil → Create repository → nombre: titanic-explorer → Public → Create.

⚠ El repositorio debe existir en Docker Hub ANTES de hacer el push.

Si no existe, recibirás el error: push access denied / repository does not exist.
Anota tu username exacto — lo necesitarás en los siguientes comandos.

PASO 11 Iniciar sesión desde WSL

```
sudo docker login
```

Introduce tu username y contraseña cuando los pida. Confirma que ves:

```
Login Succeeded
```

Si ves 'Login Succeeded' → continúa.

Si da error → cierra sesión primero: sudo docker logout y vuelve a intentarlo.
Verifica que el username es exactamente igual al de Docker Hub (mayúsculas incluidas).

PASO 12 Etiquetar la imagen con tu username

Las imágenes en Docker Hub siguen el formato: `usuario/nombre:version`

Etiqueta la imagen con tu username — igual que vincular un repo local a GitHub:

```
sudo docker tag titanic-explorer TU_USUARIO/titanic-explorer:v1.0
```

Verifica que las dos etiquetas existen (apuntan a la misma imagen):

```
sudo docker images | grep titanic
```

titanic-explorer	latest	0893d1...	982MB
TU_USUARIO/titanic-explorer	v1.0	0893d1...	982MB

El mismo ID (0893d1...) en ambas líneas confirma que son la misma imagen.

Si el tag tiene un username incorrecto, elimínalo y crea uno nuevo:

```
sudo docker rmi nombre-incorrecto/titanic-explorer:v1.0
```

PASO 13 Subir la imagen a Docker Hub (hacer el push)

```
sudo docker push TU_USUARIO/titanic-explorer:v1.0
```

Verás cómo se suben las capas una a una:

```
The push refers to repository [docker.io/TU_USUARIO/titanic-explorer]
v1.0: digest: sha256:abc123... size: 1234
```

✓ **Cuando termine, ve a <https://hub.docker.com> → tu perfil.**

Verás el repositorio titanic-explorer publicado y público.

Cualquier persona puede descargarlo desde ese momento.

PARTE 7 — Explorar el Interior del Contenedor

Una vez que tienes el contenedor corriendo, puedes entrar dentro y ver exactamente qué hay — como abrir la caja y mirar adentro.

Opción A — Entrar en modo interactivo

```
sudo docker run -it titanic-explorer bash
```

Esto abre una terminal dentro del contenedor. Puedes navegar como en cualquier Linux:

ls /app	# ver los ficheros del proyecto
cat /app/app.py	# ver el código fuente
cd /app/pages	# entrar a la carpeta pages
ls -la	# ver todos los archivos
exit	# salir del contenedor

Momento clave:

Dentro del contenedor solo está lo que pusiste en el Dockerfile — ni más ni menos. No hay venv/, no hay archivos de Windows, no hay nada extra. Eso es exactamente lo que hace que Docker sea predecible y portable.

Opción B — Copiar ficheros del contenedor a tu máquina

```
# Primero levanta el contenedor en segundo plano
sudo docker run -d --name titanic-app titanic-explorer

# Copia lo que quieras a tu máquina local
docker cp titanic-app:/app/app.py ./app.py
docker cp titanic-app:/app/pages ./pages
```

Opción C — Ver la metadata de la imagen

```
docker inspect titanic-explorer
```

Muestra toda la información de la imagen: capas, variables de entorno, comando de arranque, puertos...

PARTE 8 — Descargar y Ejecutar desde Otra Máquina

Este es el resultado final. Desde cualquier máquina con Docker instalado — sin Python, sin Streamlit, sin nada más:

```
docker pull TU_USUARIO/titanic-explorer:v1.0
docker run -p 8501:8501 TU_USUARIO/titanic-explorer:v1.0
```

Abrir navegador en `http://localhost:8501` → app funcionando. ✓

O en un solo comando (si no tiene la imagen, la descarga automáticamente):

```
docker run -p 8501:8501 TU_USUARIO/titanic-explorer:v1.0
```

Resumen del flujo completo

1 Tu código — app.py + pages/ + requirements.txt

↓ `docker build -t titanic-explorer .`

2 Imagen local — titanic-explorer:latest

↓ `docker run -p 8501:8501 titanic-explorer`

3 Contenedor en ejecución — `http://localhost:8501` 🎉

↓ `docker tag + docker push`

4 Docker Hub — imagen pública — accesible globalmente

↓ `docker pull + docker run` → *desde cualquier máquina*

5 Misma app, en cualquier lugar 🌍 — sin instalar Python ni dependencias

Referencia Rápida — Comandos del Día

Qué hace	Comando
Construir la imagen	<code>sudo docker build -t titanic-explorer .</code>
Ver imágenes locales	<code>sudo docker images</code>
Ejecutar el contenedor	<code>sudo docker run -p 8501:8501 titanic-explorer</code>
Ejecutar en segundo plano	<code>sudo docker run -d -p 8501:8501 --name app titanic-explorer</code>
Ver contenedores activos	<code>sudo docker ps</code>
Ver TODOS los contenedores	<code>sudo docker ps -a</code>
Parar un contenedor	<code>sudo docker stop <nombre_o_id></code>
Entrar dentro del contenedor	<code>sudo docker run -it titanic-explorer bash</code>
Copiar fichero del contenedor	<code>docker cp nombre-app:/app/app.py ./app.py</code>
Ver metadata de la imagen	<code>docker inspect titanic-explorer</code>
Iniciar sesión en Docker Hub	<code>sudo docker login</code>
Etiquetar para Docker Hub	<code>sudo docker tag titanic-explorer USER/titanic-explorer:v1.0</code>
Subir imagen a Docker Hub	<code>sudo docker push USER/titanic-explorer:v1.0</code>
Descargar imagen de Docker Hub	<code>docker pull USER/titanic-explorer:v1.0</code>
Eliminar un contenedor	<code>sudo docker rm <nombre_o_id></code>
Eliminar una imagen	<code>sudo docker rmi titanic-explorer</code>

Referencia Rápida — Todos los Comandos

Docker vs Git — comandos equivalentes

Git	Docker	¿Qué hace?
git status	docker images	Ver qué tienes en local
git commit -m 'msg'	docker build -t mi-app .	Guardar el estado actual
git remote add origin ...	docker tag mi-app user/mi-app:v1.0	Vincular con el remoto
git push origin main	docker push user/mi-app:v1.0	Subir al repositorio remoto
git pull	docker pull user/mi-app:v1.0	Descargar desde el remoto
git log --oneline	docker images	Ver versiones disponibles

Comandos completos del reto

Fase	Comando
Crear .dockerignore	cat > .dockerignore << 'EOF' ... EOF
Verificar archivos	ls -la
Construir imagen	sudo docker build -t mi-app-streamlit .
Ver imágenes locales	sudo docker images
Probar en local	sudo docker run -p 8501:8501 mi-app-streamlit
Ver contenedores activos	sudo docker ps
Parar el contenedor	Ctrl + C / sudo docker stop <id>
Iniciar sesión Docker Hub	sudo docker login
Etiquetar la imagen	sudo docker tag mi-app-streamlit USER/mi-app-streamlit:v1.0
Verificar etiqueta	sudo docker images grep mi-app
Subir a Docker Hub	sudo docker push USER/mi-app-streamlit:v1.0
URL del repositorio	https://hub.docker.com/r/USER/mi-app-streamlit